Lifecycle State Consistency Control Module (LSCCM)

Lifecycle State Consistency Control Module (LSCCM)

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Parent Standard: Software Lifecycle Control Standard (SLCS)

Category: Governance & Enforcement

Subcategory: Software Lifecycle Control Governance

Type: Core Module

Governed Space: Software Lifecycle State Consistency Governance

Module ID: SLGA-SLCS-LSCCM-01

Origin ID: OOF-SLGA-SLCS-LSCCM

Version: 1.0

Status: Canonical Open

Effective Date: 15.10.2026

Compatibility: Methodology OS — Modular Compatibility

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: Protected Methodological Work

Canonical Language: UCL — Universal Canonical Language (English)

Canonical Definition

Lifecycle State Consistency Control Module (LSCCM) is the Core Module within Software Lifecycle Control Standard (SLCS) that governs how declared, recorded, observed, and verified lifecycle states of identifiable software objects are established, compared, reconciled, and maintained consistently across software versions, configurations, environments, operational conditions, and governance domains.

LSCCM establishes lifecycle state consistency as an evidence-supported

relationship between what a software object is declared to be, what its actual technical and operational conditions demonstrate, and which lifecycle governance requirements apply to that condition.

The module distinguishes software identity from lifecycle state,

administrative classification from operational reality, recorded state from verified state, and legitimate multidimensional lifecycle conditions from contradictory or unsupported state claims.

It requires material lifecycle state assertions to remain associated with

identifiable software objects, applicable versions, configurations, environments, effective times, evidence sources, verification methods, responsible functions, and known limitations.

LSCCM recognizes that software may simultaneously possess different states across distinct lifecycle dimensions. A software version may be released but not deployed, deployed but restricted, operational but unsupported, or retired in one environment while remaining active in another.

Such conditions do not necessarily constitute inconsistency.

The module therefore governs the interpretation and reconciliation of

lifecycle states according to their actual scope and meaning rather than requiring every software object to possess one simplified universal status.

LSCCM identifies conflicting, incomplete, outdated, unsupported, or

incorrectly generalized lifecycle claims and establishes the requirements for their assessment and reconciliation.

It does not independently authorize lifecycle transitions, approve exceptions, transfer lifecycle control, or determine the operational effectiveness of all lifecycle governance controls.

Its principal governance output is the Software Lifecycle State Consistency Profile.

Operational Role

LSCCM establishes the governance mechanism for maintaining consistency

between software lifecycle state claims and actual software conditions.

It governs:

• Software lifecycle state identity

• Software object identification

• State source identification

• Declared lifecycle states

• Recorded lifecycle states

• Observed lifecycle conditions

• Verified lifecycle states

• Lifecycle state dimensions

• State scope

• Version-specific states

• Configuration-specific states

• Environment-specific states

• Time-specific states

• State evidence

• State applicability

• State consistency

• State contradictions

• State conflicts

• State uncertainty

• State incompleteness

• State reconciliation

• State correction

• State validity

• State continuity

• Cross-standard state alignment

• State change recognition

• State reassessment

• State verification

• State classification

• State governance outputs

LSCCM establishes whether lifecycle state claims are consistent with applicable evidence and operational reality.

It does not independently determine whether a software transition should be authorized.

Operational Question

How can the lifecycle state of an identifiable software object remain consistent, evidence-supported, and verifiable when different systems, actors, environments, versions, and governance domains may describe the same software through incomplete, outdated, conflicting, or legitimately different state claims?

Position in Architecture Governance Flow

SLGA™ Governance Flow

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-Cutting Governance Layer

Software Lifecycle Control Standard (SLCS)

SLCS Core Module Flow

Lifecycle State Consistency Control Module (LSCCM) → Lifecycle Transition

Control Module (LTCM) → Lifecycle Exception Management Module (LEMM) → Lifecycle Control Transfer & Continuity Module (LCTCM) → Lifecycle Control Assessment Module (LCAM)

LSCCM is the first Core Module within SLCS.

It establishes the lifecycle state consistency conditions required by the

other SLCS modules.

It provides evidence-supported state information for transition control,

exception assessment, control continuity, and lifecycle control evaluation.

Its governance function remains applicable throughout the software

lifecycle.

• Governed Space

Software Lifecycle State Consistency Governance

This governed space concerns the identification, interpretation,

verification, comparison, and reconciliation of software lifecycle states.

It begins when an identifiable software object is associated with a material lifecycle state claim or observable lifecycle condition.

It includes the scope, source, meaning, evidence, validity, consistency,

and resulting classification of lifecycle states.

It remains applicable where state claims are incomplete, conflicting, outdated, uncertain, or subject to reassessment.

It also applies where multiple lifecycle states legitimately coexist across different software dimensions.

The governed space continues as long as material lifecycle state

relationships require governance.

Governed Object

The primary Governed Object is:

A Software Lifecycle State Consistency Case representing the

evidence-supported relationship between an identifiable software object, one or more lifecycle state claims, their applicable dimensions and scopes, observed operational conditions, supporting evidence, identified inconsistencies,

reconciliation requirements, and resulting verified lifecycle state classification.

The governed object must preserve, where applicable:

State Consistency Case ID

Software Object ID

Software Version

Build Reference

Configuration Reference

Environment Reference

Lifecycle Dimension

Declared State

Recorded State

Observed Condition

Verified State

State Source

State Effective Time

Evidence Reference

Verification Method

Consistency Assessment

Conflict Status

Reconciliation Requirement

Responsible Function

Resulting State Classification

Record Version

Scope

LSCCM governs:

Lifecycle State Identification

Lifecycle State Source Recognition

Declared State Assessment

Recorded State Assessment

Observed State Assessment

Verified State Assessment

Lifecycle State Dimension Classification

Lifecycle State Scope

Version and Configuration Applicability

Environment Applicability

Temporal Applicability

Lifecycle State Evidence

Lifecycle State Comparison

Lifecycle State Conflict Recognition

Lifecycle State Reconciliation

Lifecycle State Correction

Lifecycle State Uncertainty

Lifecycle State Validity

Cross-Standard State Consistency

Lifecycle State Reassessment

Lifecycle State Governance Classification

LSCCM does not prescribe a universal lifecycle state taxonomy for all software.

State models must reflect the governed software and applicable lifecycle requirements.

Foundational Principle

A software lifecycle state is legitimate only within the scope, context, time, and evidence that establish it; no recorded state may be generalized beyond those conditions without appropriate verification.

A software object may be released without being deployed.

A deployed software instance may be inactive.

An operational software version may be unsupported.

A software product may be retired in one environment while remaining active in another.

These conditions must be interpreted according to their actual meaning.

Recorded State ≠ Verified State.

Different State Dimensions ≠ State Conflict.

Lifecycle State Case Identity

Every material state consistency assessment must have a stable identity.

Relevant elements include:

State Consistency Case ID

Software Object Reference

Lifecycle Dimension

State Scope

Assessment Origin

Responsible Function

Assessment Time

Evidence Reference

Assessment Status

Record Version

The identity must remain stable across verification, conflict assessment, reconciliation, and reassessment.

Software Object Identification

LSCCM must identify the software object to which a state claim applies.

Relevant elements include:

Software Product

Software Component

Software Version

Build

Artifact

Configuration

Deployment Instance

Environment

Service Identity

Operational Function

A state claim must not automatically apply to every software object sharing the same product name.

Software Identity Granularity

Lifecycle state consistency depends on the appropriate level of software

identity.

Potential identity levels include:

Product

Application

Service

Component

Version

Build

Artifact

Deployment Instance

Runtime Process

Embedded Software Instance

LSCCM must establish the granularity relevant to the governance decision.

Lifecycle State Claim

A lifecycle state claim is an assertion concerning the condition of an identifiable software object.

Relevant elements include:

Claim ID

Software Identity

Claimed State

State Dimension

Applicable Scope

Claim Origin

Claim Time

Effective Time

Supporting Evidence

Known Limitations

A claim may be accurate, incomplete, outdated, unsupported, or conflicting.

Declared Lifecycle State

A declared lifecycle state is a status formally asserted by an actor, organization, or governance mechanism.

Potential examples include:

Development Completed

Release Approved

Production Ready

Operational

Unsupported

Retired

A declared state must remain distinguishable from its verification.

Recorded Lifecycle State

A recorded lifecycle state is a status preserved within an information system or governance record.

Potential sources include:

Software Asset Register

Development Platform

Release Management System

Deployment System

Service Management Platform

Configuration Database

Retirement Register

The existence of a record does not independently establish that the recorded condition is current or accurate.

Observed Lifecycle Condition

An observed lifecycle condition is a technical or operational condition established through an observation process.

Potential observations include:

Artifact Exists

Software Is Deployed

Process Is Executing

Service Is Responding

Users Are Active

Interface Is Available

Scheduled Task Is Running

Software Is Not Observed Executing

Observation must remain associated with its method, coverage, and time.

Verified Lifecycle State

A verified lifecycle state is a state conclusion supported by evidence sufficient for the applicable governance purpose.

Relevant elements include:

Software Identity

State Dimension

Applicable Scope

Evidence

Verification Method

Verification Time

Known Limitations

Resulting State

Verification must not claim certainty beyond the evidence.

Lifecycle State Dimensions

LSCCM recognizes multiple lifecycle dimensions.

Potential dimensions include:

Requirements State

Creation State

Dependency State

Configuration State

Quality State

Security State

Release State

Deployment State

Operational State

Support State

Change State

Retirement State

These dimensions may legitimately contain different classifications at the same time.

State Dimension Separation

LSCCM must distinguish state dimensions that govern different realities.

For example:

Release Authorized

does not establish:

Production Deployed

And:

Production Deployed

does not establish:

Operationally Available

Similarly:

Support Terminated

does not establish:

Software Retired

State dimensions must remain independently identifiable.

Lifecycle State Scope

Every material lifecycle state claim must have an identifiable scope.

Relevant dimensions include:

Software Object

Version

Build

Configuration

Environment

Deployment Instance

Operational Function

Organizational Unit

Geographic Scope

Time

A state valid for one scope must not automatically be generalized to another.

Version-Specific State

Different versions of the same software may occupy different lifecycle states.

Potential conditions include:

Version A — Operational

Version B — Release Candidate

Version C — Under Development

Version D — Retired

LSCCM must preserve version-specific state relationships.

Configuration-Specific State

Lifecycle state conclusions may depend on configuration.

Relevant elements include:

Configuration Baseline

Feature Settings

Security Settings

Environment Variables

Dependency Versions

Deployment Parameters

Runtime Conditions

A verified state for one configuration must not automatically apply to another materially different configuration.

Environment-Specific State

Software may possess different states across environments.

Potential environments include:

Development

Testing

Staging

Production

Disaster Recovery

Cloud Region

Edge Deployment

Embedded Device

Archived Environment

LSCCM must preserve environment-specific applicability.

Temporal State Applicability

Lifecycle state claims must remain associated with their relevant time.

Potential elements include:

Observation Time

Record Creation Time

Effective Time

Verification Time

Expiration Time

Reassessment Time

A previously verified state may no longer describe the current software condition.

State Source Identification

LSCCM must identify the origin of material lifecycle state information.

Potential sources include:

Human Declaration

Organizational Approval

Software Inventory

Deployment Record

Runtime Monitoring

Configuration Record

Security Assessment

Release Record

Retirement Evidence

Source identification supports traceability but does not automatically establish source reliability.

State Source Reliability

LSCCM must evaluate the reliability of state sources where material to the governance conclusion.

Relevant considerations include:

Source Authority

Observation Capability

Data Accuracy

Update Frequency

Coverage

Known Limitations

Integrity

Independence Where Required

A technically authoritative source may still contain outdated or incomplete information.

State Evidence

Lifecycle state evidence may include:

Artifact Records

Build Records

Configuration Baselines

Test Results

Security Assessments

Release Authorizations

Deployment Records

Runtime Logs

Monitoring Results

Support Notices

Retirement Verification

Evidence must remain connected to the state claim it supports.

Evidence Applicability

Evidence applicability must be assessed against:

Software Identity

Version

Build

Configuration

Environment

State Dimension

Time

Verification Purpose

Known Limitations

Evidence from a different software condition must not automatically establish the current state.

Evidence Sufficiency

LSCCM must evaluate whether available evidence is sufficient for the intended state conclusion.

Relevant considerations include:

Evidence Relevance

Evidence Coverage

Evidence Integrity

Observation Reliability

Temporal Validity

Conflicting Evidence

Material Uncertainty

Required Verification Criteria

Evidence sufficiency is contextual rather than universal.

State Comparison

LSCCM must compare relevant lifecycle state claims.

Potential comparisons include:

Declared vs Recorded

Recorded vs Observed

Declared vs Verified

Current vs Historical

Source System vs Destination System

Release vs Deployment

Operational vs Retirement

Support vs Operational Use

Comparison must preserve differences in scope and state dimension.

State Compatibility

Different state claims may be compatible even when their labels differ.

Examples include:

Released and Not Deployed

Deployed and Restricted

Operational and Unsupported

Retired in Production and Retained for Recovery

Change Active and Operational

LSCCM must determine whether apparent differences represent legitimate coexistence.

State Contradiction

A state contradiction occurs when claims concerning the same governed condition cannot both be valid under their applicable meaning, scope, and time.

Potential examples include:

Same Instance Declared Removed and Verified Running

Same Build Declared Approved and Recorded as Rejected

Same Deployment Declared Inactive and Verified Processing Requests

Same Software Scope Declared Fully Retired While Required Active Execution

Remains

Contradiction assessment must establish the actual shared scope before classification.

State Conflict Recognition

LSCCM must recognize material state conflicts.

Relevant elements include:

Conflicting Claims

Affected Software

Shared Scope

Relevant Time

Evidence Sources

Operational Consequences

Responsible Functions

Required Reconciliation

Conflict recognition must not automatically determine which claim is correct.

Apparent State Conflict

An apparent conflict may arise from differences in terminology, granularity, environment, or time.

Potential examples include:

Product Retired vs Version Operational

Service Disabled vs Recovery Copy Retained

Release Completed vs Deployment Pending

Support Ended vs Software Still Operating

LSCCM must distinguish apparent conflicts from genuine contradictions.

Genuine State Conflict

A genuine conflict exists where material claims remain incompatible after their scope, meaning, and temporal applicability have been reconciled.

Relevant elements include:

Common Software Identity

Common State Dimension

Common Scope

Overlapping Time

Incompatible Conditions

Supporting Evidence

Required Governance Response

Genuine conflicts must remain unresolved until sufficient evidence establishes an appropriate conclusion.

State Incompleteness

A lifecycle state may be incomplete where material dimensions or conditions are missing.

Potential examples include:

Deployment State Without Environment

Release State Without Artifact Identity

Retirement State Without Scope

Operational State Without Observation Time

Support State Without Provider or Version

Incomplete state claims must not be generalized beyond their established meaning.

Outdated State

A previously valid state may become outdated after material changes.

Potential triggers include:

New Software Version

Configuration Change

Deployment Change

Support Termination

Security Event

Operational Failure

Migration

Retirement

Outdated records must remain historically traceable while being distinguished from current conditions.

State Uncertainty

LSCCM must preserve uncertainty where a lifecycle state cannot be established.

Potential conditions include:

Unknown Deployment

Unverified Runtime State

Missing Configuration Identity

Conflicting Monitoring Results

Incomplete Retirement Evidence

Uncertain Support Condition

Unknown Active Instance

Insufficient Evidence

Uncertainty must not be converted into a definitive state without appropriate verification.

State Confidence and Limitations

Where confidence assessments are used, LSCCM must preserve their basis.

Relevant elements include:

Evidence Coverage

Observation Reliability

Known Blind Spots

Temporal Validity

Verification Method

Conflicting Evidence

Unresolved Assumptions

Confidence must not substitute for mandatory verification requirements.

State Reconciliation

State reconciliation establishes an evidence-supported relationship between

conflicting or incomplete lifecycle state claims.

Relevant steps include:

Identify Claims

Confirm Software Identity

Align Scope

Align Time

Separate State Dimensions

Evaluate Evidence

Determine Compatibility or Conflict

Establish Verified Conclusion

Preserve Unresolved Conditions

Reconciliation must not erase the history of earlier claims.

State Reconciliation Authority

LSCCM must identify authority for material state corrections or reconciliations.

Relevant elements include:

Responsible Function

Applicable Governance Authority

Evidence Review Responsibility

State Correction Permission

Escalation Requirements

Approval Conditions

Record Integrity Requirements

Technical ability to edit a lifecycle record does not establish authority to determine the underlying governance state.

State Correction

State correction updates an inaccurate, incomplete, or outdated lifecycle

record.

Relevant elements include:

Original State Claim

Correction Reason

Supporting Evidence

Corrected State

Responsible Actor

Applicable Authority

Correction Time

Historical Record

Correction must preserve the distinction between changing a record and changing the actual software condition.

State Change Recognition

LSCCM must recognize material changes to lifecycle conditions.

Potential triggers include:

Artifact Creation

Build Change

Configuration Change

Release Decision

Deployment

Operational Activation

Support Change

Security Restriction

Migration

Retirement

Recognition of a state change does not independently authorize the underlying transition.

State Change Verification

Where a state change is material, LSCCM must establish the resulting state through applicable evidence.

Relevant elements include:

Previous State

Change Event

Intended State

Observed State

Verification Evidence

Resulting Classification

Known Limitations

Transition authorization and execution remain governed by LTCM and

applicable specialized standards.

State Continuity

LSCCM must preserve the relationship between successive lifecycle state records.

Relevant elements include:

Previous State

Current State

Change Origin

Effective Time

Evidence

Applicable Software Identity

Record Version

Unresolved Differences

State continuity must support historical reconstruction without assuming that every transition was legitimate.

Cross-Standard State Alignment

LSCCM must assess consistency between state information produced by different SLGA™ Parent Standards.

Relevant relationships include:

Requirements and Creation

Creation and Configuration

Configuration and Quality

Quality and Security

Security and Release

Release and Deployment

Deployment and Change Management

Change Management and Retirement

State alignment must preserve the independent meaning of each specialized governance domain.

Release–Deployment State Alignment

LSCCM must distinguish:

Release Candidate

Release Authorized

Deployment Authorized

Deployment Executed

Production Operational

These states may occur at different times and under different authority conditions.

Release authorization does not establish that the approved artifact is deployed.

Deployment–Operational State Alignment

LSCCM must distinguish:

Artifact Deployed

Service Started

Service Available

Service Operational

Service Restricted

Service Failed

A deployed artifact may exist without satisfying operational availability or functionality requirements.

Support–Operational State Alignment

LSCCM must distinguish:

Supported Software

Limited Support

Security-Only Support

Unsupported Software

Operational Software

Inactive Software

Retired Software

Support conditions and operational states must remain independently identifiable.

Change–Lifecycle State Alignment

Software changes may affect multiple lifecycle dimensions.

Relevant elements include:

Change Identity

Affected Software

Previous State

Changed Configuration

Required Reassessment

Resulting State

Verification Evidence

LSCCM must preserve consistency between change outcomes and resulting lifecycle state claims.

Retirement–Operational State Alignment

LSCCM must assess retirement-related state claims against actual software conditions.

Relevant elements include:

Retirement Scope

Declared Retirement Status

Deployment State

Execution State

Residual Dependencies

Authority State

Continuing Obligations

Retirement Closure Evidence

Administrative retirement status must not automatically establish verified operational retirement.

Multi-Environment State Consistency

Software may exist simultaneously across multiple environments.

LSCCM must preserve:

Environment Identity

Software Version

Configuration

Deployment State

Operational State

Support Condition

Applicable Restrictions

Evidence

State consistency must not require identical states across environments where differences are legitimate.

Distributed Software State Consistency

Distributed systems may contain multiple components or instances with different lifecycle conditions.

Relevant elements include:

Component Identity

Instance Identity

Deployment Location

Version

Operational State

Dependency Relationship

Transition Status

Verification Evidence

A system-level lifecycle state must not conceal material differences among its components.

Cloud and Ephemeral Software States

Cloud-native and ephemeral environments may create rapidly changing software instances.

Potential objects include:

Containers

Serverless Functions

Autoscaled Services

Temporary Environments

Short-Lived Workloads

Distributed Deployments

LSCCM must preserve sufficient identity and temporal evidence to establish relevant lifecycle conditions.

A short-lived instance must not be excluded from governance solely because it no longer exists when an assessment occurs.

Autonomous Software State Consistency

Autonomous systems may modify software environments or trigger lifecycle activities.

Relevant elements include:

Autonomous Actor Identity

Affected Software

Observed Change

Declared State

Actual State

Applicable Authority

Execution Evidence

Verification Evidence

LSCCM establishes the resulting state consistency without independently

legitimizing the autonomous action.

Lifecycle State Monitoring

State monitoring may identify material changes or inconsistencies.

Potential monitoring sources include:

Deployment Platforms

Runtime Monitoring

Configuration Systems

Software Inventories

Security Monitoring

Release Systems

Support Registers

Retirement Registers

Monitoring must remain associated with identifiable coverage and limitations.

State Monitoring Gaps

LSCCM must identify material monitoring gaps.

Potential gaps include:

Unmonitored Environment

Unknown Software Instance

Missing Runtime Visibility

Incomplete Configuration Records

Unobserved External Deployment

Delayed State Updates

Missing Historical Records

Monitoring gaps must not be interpreted as evidence that no conflicting condition exists.

State Reassessment

LSCCM must support reassessment when:

New Evidence Appears

Software Identity Changes

Version Changes

Configuration Changes

Deployment Conditions Change

Support Conditions Change

Lifecycle Transition Occurs

State Conflict Is Discovered

Retirement Is Reopened

Reassessment must preserve earlier conclusions and their evidence.

State Classification

LSCCM recognizes the following governance classifications:

State Claim Identified

State Assessment Pending

Declared State

Recorded State

Observed State

Verified State

Consistent

Conditionally Consistent

Compatible Multidimensional States

Apparent Conflict

Confirmed Conflict

Incomplete State

Outdated State

Unverified State

Reconciliation Pending

Reconciled

Correction Required

Reassessment Required

Unknown

Conflicted

Each classification must remain associated with its applicable scope, evidence, and time.

Governance Requirements

A conforming LSCCM implementation must satisfy the following requirements. Software Identity and State Claims

• Establish a stable state consistency case identity

• Identify the software object

• Establish appropriate identity granularity

• Identify material lifecycle state claims

• Preserve claim origins

• Establish state dimensions

• Establish state scope

• Preserve version applicability

• Preserve configuration applicability

• Preserve environment applicability

Evidence and Verification

• Identify material state evidence

• Establish evidence applicability

• Evaluate evidence sufficiency

• Preserve observation methods

• Preserve temporal applicability

• Distinguish recorded and verified states

• Identify evidence limitations

• Preserve material uncertainty

Consistency and Conflict

• Compare relevant state claims

• Distinguish compatible multidimensional states

• Identify apparent conflicts

• Identify genuine contradictions

• Preserve incomplete state claims

• Identify outdated state information

• Establish conflict significance

• Preserve unresolved conflicts

• Prevent state generalization beyond verified scope

Reconciliation and Correction

• Establish reconciliation requirements

• Identify competent reconciliation authority

• Preserve reconciliation evidence

• Establish verified state conclusions

• Preserve original state claims

• Govern material state corrections

• Maintain historical state continuity

• Identify reassessment triggers

Cross-Standard Continuity

• Maintain release–deployment state distinctions

• Maintain deployment–operational state distinctions

• Maintain support–operational state distinctions

• Maintain change–lifecycle state distinctions

• Maintain retirement–operational state distinctions

• Preserve multi-environment state differences

• Preserve distributed software state differences

• Identify material monitoring gaps

• Support automated state assessment within legitimate authority

• Provide verified state information to subsequent SLCS modules

Governance Logic

Software Object Identification → State Consistency Case Identity → State

Claim Identification → State Source → State Dimension → State Scope → Temporal Applicability → Evidence Collection → Observed Condition → State Comparison → Compatibility Assessment → Conflict Recognition → Evidence Sufficiency → Reconciliation → Verified State Classification → State Continuity → Software Lifecycle State Consistency Profile

Where states are consistent:

Declared State → Applicable Evidence → Observed Condition → Verification →

Consistent State

Where states differ legitimately:

Multiple State Claims → Dimension and Scope Separation → Compatibility

Assessment → Compatible Multidimensional States

Where an apparent conflict exists:

Conflicting Labels → Identity and Scope Reconciliation → Temporal

Assessment → Compatible or Confirmed Conflict

Where a genuine conflict exists:

Incompatible State Claims → Evidence Assessment → Reconciliation

Requirements → Corrected or Unresolved State

Where evidence is insufficient:

State Claim → Evidence Gap → Unverified State → Further Assessment

Where a previously verified state becomes outdated:

Verified Historical State → Material Change → Reassessment → Updated State

Classification

LSCCM must not treat a lifecycle state record as proof of the actual software condition without appropriate verification.

Governance Outputs

The principal output is:

Software Lifecycle State Consistency Profile

Supporting outputs may include:

Lifecycle State Consistency Case Identity

Software Lifecycle Identity Record

Lifecycle State Claim Register

Lifecycle State Dimension Profile

Lifecycle State Scope Record

Lifecycle State Evidence Register

Lifecycle State Verification Record

Lifecycle State Compatibility Assessment

Lifecycle State Conflict Record

Lifecycle State Uncertainty Record

Lifecycle State Reconciliation Record

Lifecycle State Correction Record

Lifecycle State Continuity Record

Cross-Standard State Alignment Record

Lifecycle State Monitoring Gap Record

Lifecycle State Reassessment Record

Minimum Implementation Framework

A minimum LSCCM implementation must establish eight governance layers. Layer 1 — Software Identity

Identify the governed software object at the appropriate product, version,

configuration, component, or deployment-instance level. Layer 2 — State Claim Identification

Collect relevant declared, recorded, observed, and previously verified

lifecycle state claims. Layer 3 — State Context

Establish the dimension, scope, environment, version, configuration, and

temporal applicability of each material claim. Layer 4 — Evidence Assessment

Identify applicable evidence, verification methods, source limitations, and

material uncertainty. Layer 5 — State Comparison

Compare state claims and distinguish legitimate multidimensional

differences from genuine contradictions. Layer 6 — Reconciliation

Establish the requirements, responsibility, authority, and evidence

necessary to reconcile material inconsistencies. Layer 7 — State Verification

Establish evidence-supported lifecycle state classifications and preserve

unresolved conditions. Layer 8 — Continuity and Reassessment

Maintain historical state records, identify change triggers, and provide

verified state information to other lifecycle governance mechanisms.

The framework may be implemented through software asset registers, configuration systems, deployment platforms, runtime monitoring, governance records, or AI-assisted analysis.

The implementation mechanism must adapt to the actual lifecycle state relationships.

Cross-Module Boundary

LSCCM → LTCM

LSCCM establishes the consistency and verification of lifecycle state

claims.

LTCM governs the legitimacy, authority, conditions, execution, and

verification of transitions between lifecycle states.

LSCCM → LEMM

LSCCM identifies lifecycle states affected by exceptions and records the

resulting state implications.

LEMM governs the legitimacy, conditions, and continuity of exceptions.

LSCCM → LCTCM

LSCCM preserves state information relevant to lifecycle control transfer.

LCTCM governs the transfer and continuity of responsibility, authority, and

control capability.

LSCCM → LCAM

LSCCM provides state consistency evidence.

LCAM assesses whether lifecycle control mechanisms are appropriately designed and operationally effective.

State consistency is a prerequisite for reliable lifecycle governance, but it does not independently establish transition legitimacy or control effectiveness.

Cross-Architecture Boundary

LSCCM governs lifecycle state consistency within SLGA™.

It does not replace specialized lifecycle governance.

SRS — Software Requirements Standard

Requirements-related state information.

SCS — Software Creation Standard

Creation and development state information.

SDS — Software Dependency Standard

Dependency condition and continuity information.

SCFS — Software Configuration Standard

Artifact, build, version, and configuration identity.

SQS — Software Quality Standard

Quality assessment and defect-related conditions.

SSS — Software Security Standard

Security assessment, exposure, and remediation conditions.

SRLS — Software Release Standard

Release candidate, readiness, authorization, and withdrawal states.

SDPS — Software Deployment Standard

Deployment, production, monitoring, and recovery states.

SCMS — Software Change Management Standard

Change and lifecycle re-entry conditions.

SRTS — Software Retirement Standard

Retirement planning, support, migration, residual dependency, and closure

states.

LSCCM may also interact with applicable OOF® identity, authority, accountability, operational reality, AI, and autonomous systems governance mechanisms.

It does not redefine their specialized governance functions.

Practical Applications

Enterprise Software Governance

Reconciling software inventory records with actual deployment and

operational conditions.

Cloud-Native Platforms

Maintaining lifecycle state consistency across distributed services,

containers, and ephemeral workloads.

Software Release Management

Distinguishing authorized releases from actual deployed artifacts.

Software Supply Chains

Preserving version-specific lifecycle conditions across components and

dependencies.

AI Systems

Distinguishing model availability, deployment, operational use,

restrictions, and retirement.

Autonomous Software Agents

Verifying lifecycle state claims after automated or autonomous system

changes.

Industrial Software

Maintaining consistency between software records, firmware versions,

deployed controllers, and physical operational conditions.

Software Retirement

Preventing administrative retirement status from substituting for verified

operational withdrawal.

Practical Use Case 1 — Conflicting Release and Production States

An organization maintains a software release register and a separate

production deployment platform.

The release register identifies version 4.2 as the currently approved

production version.

A runtime assessment identifies that several production instances are executing version 4.1, while other instances are executing version 4.2 with different configurations.

LSCCM evaluates:

Software Identity → Version → Configuration → Environment → Declared

Release State → Observed Deployment State → Evidence → Consistency Assessment

The assessment establishes that the release register describes an authorized release condition but does not accurately represent the complete production deployment state.

LSCCM preserves:

Version 4.2 — Release Authorized

Version 4.2 — Partially Deployed

Version 4.1 — Still Operational in Identified Instances

Configuration Differences — Verification Required

The resulting governance classification identifies a cross-system lifecycle

state inconsistency.

The applicable deployment and configuration governance mechanisms determine

the required corrective actions.

Approved Production Version ≠ Verified Version Across Production.

Practical Use Case 2 — AI Service Declared Retired While Still Invoked

An organization replaces an AI inference service with a newer model.

The software inventory identifies the previous service as retired.

However, operational monitoring detects occasional requests to the legacy

endpoint from an automated workflow.

LSCCM evaluates:

Service Identity → Declared Retirement State → Endpoint Identity →

Observed Requests → Operational Scope → Verification Evidence → State Reconciliation

The assessment identifies:

• The primary application has migrated to the replacement service

• The legacy service remains technically accessible

• An automated workflow continues invoking the original endpoint

• The inventory retirement status does not accurately describe the

complete operational condition.

LSCCM classifies the lifecycle state as:

Retirement Declared — Operational Withdrawal Not Fully Verified.

The relevant retirement and dependency governance mechanisms determine the

required resolution.

Service Declared Retired ≠ Service Operationally Withdrawn.

Critical Distinctions

Software Identity ≠ Lifecycle State.

Product State ≠ Every Version State.

Version State ≠ Every Deployment State.

Recorded State ≠ Verified State.

Declared State ≠ Actual Operational Condition.

Observed Activity ≠ Authorized Activity.

Different Lifecycle Dimensions ≠ State Conflict.

Different Environments ≠ Inconsistent Governance.

Apparent Conflict ≠ Genuine Contradiction.

Historical State ≠ Current State.

Evidence Available ≠ Evidence Applicable.

Evidence Applicable ≠ Evidence Sufficient.

State Correction ≠ Operational State Change.

State Change Recognized ≠ Transition Authorized.

Release Authorized ≠ Deployment Verified.

Deployment Present ≠ Operationally Available.

Support Terminated ≠ Software Retired.

Software Retired Administratively ≠ Operational Withdrawal Verified.

Monitoring Silence ≠ Verified Inactivity.

State Reconciled ≠ Every Lifecycle Control Effective.

Canonical Boundary

Lifecycle State Consistency Control Module (LSCCM) governs the

identification, interpretation, comparison, verification, reconciliation, and continuity of software lifecycle state claims across identifiable software objects, versions, configurations, environments, lifecycle dimensions, and relevant times.

It establishes whether declared, recorded, and observed states are mutually compatible and supported by sufficient evidence.

It distinguishes legitimate multidimensional conditions from material

contradictions.

It preserves uncertainty where the actual lifecycle state cannot be established.

It does not independently authorize lifecycle transitions, approve exceptions, transfer control authority, or determine the effectiveness of every lifecycle governance mechanism.

Its principal governance output is the:

Software Lifecycle State Consistency Profile

This output establishes which lifecycle states are claimed, what evidence supports them, which differences are legitimate, which conflicts remain unresolved, and what state conclusions can be relied upon.

Canonical Closing Statement

Software cannot be reliably governed when its lifecycle state depends on which system, department, or record is consulted.

A release register may identify one approved version while production executes another.

A deployment platform may show an application as active while the service is unavailable.

A support register may classify software as unsupported while it continues performing essential operations.

A retirement register may identify a service as retired while external systems still invoke it.

And a distributed application may legitimately contain multiple versions, configurations, and lifecycle states simultaneously.

These conditions demonstrate that lifecycle state consistency cannot be established through a single administrative label.

LSCCM establishes the governance mechanism required to preserve the actual

meaning of software lifecycle states.

It connects state claims to identifiable software objects.

It preserves version, configuration, environment, and temporal

applicability.

It distinguishes declared conditions from observed and verified conditions.

It identifies contradictions without misclassifying legitimate

multidimensional differences.

It requires reconciliation to remain evidence-supported.

And it maintains continuity between historical and current state

conclusions.

A lifecycle state is not trustworthy because it is recorded. It becomes trustworthy when its meaning, scope, and operational reality can be verified.

Lifecycle State Consistency Control Module (LSCCM) establishes the

evidence-supported foundation of software lifecycle control—ensuring that every material lifecycle state remains identifiable, interpretable, and consistent with the software reality it claims to represent.